Instituto Politécnico Nacional

Unidad Profesional Interdisciplinaria en

Ingeniería y Tecnologías Avanzadas

Alumnas:

Hernández Arteaga Itzel

García Ortiz Martha Lesly

Profesor: De La Cruz Sosa Carlos

Carrera: Telemática

Materia: Bases de Datos Distribuidas

Grupo: 3TM2

Practica 2: Plan de ejecución

Fecha: 23/03/2026

Consulta 1:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
1	2026-03-23	134	125	303	12(Index Scan) -- Operadores con costo	Reescribir la consulta

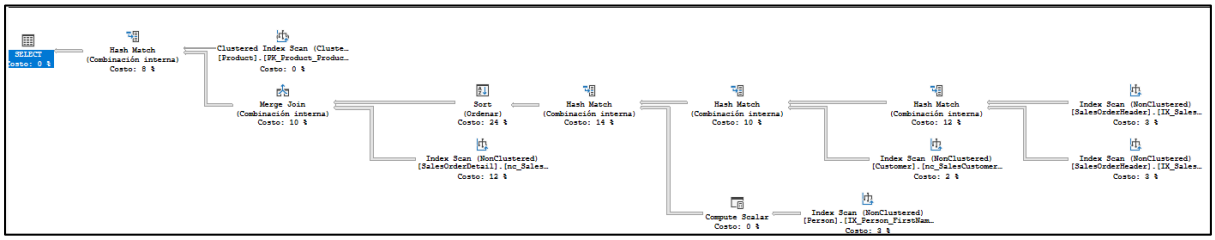


Fig.1 Plan de Ejecución estimado

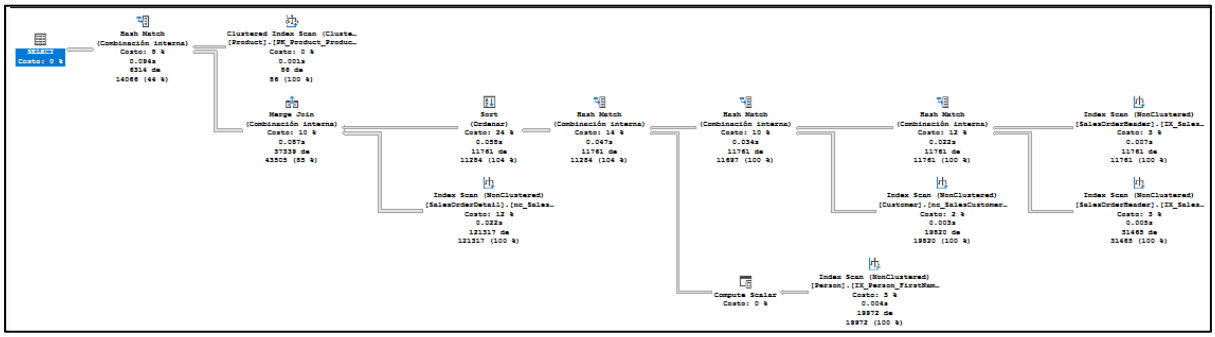


Fig.2 Plan de ejecución real

Consulta reescrita

```
SELECT  p.Name AS Producto, sod.OrderQty, soh.OrderDate,
   per.FirstName + ' ' + per.LastName AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
JOIN Person.Person per ON c.PersonID = per.BusinessEntityID
WHERE soh.OrderDate >= '2014-01-01'
  AND soh.OrderDate < '2015-01-01';
```

En esta consulta, la principal mejora fue el modificar la condición sobre la fecha. Al reemplazarla por un rango de fechas se permitió que el motor de base de datos utilizara operaciones más eficientes en el plan de ejecución.En cuanto a las lecturas lógicas, no se observó una disminución significativa. Aunque se consideró la creación de índices adicionales, estos no generaron una mejora relevante en este aspecto e incluso incrementaron el costo estimado de la consulta.

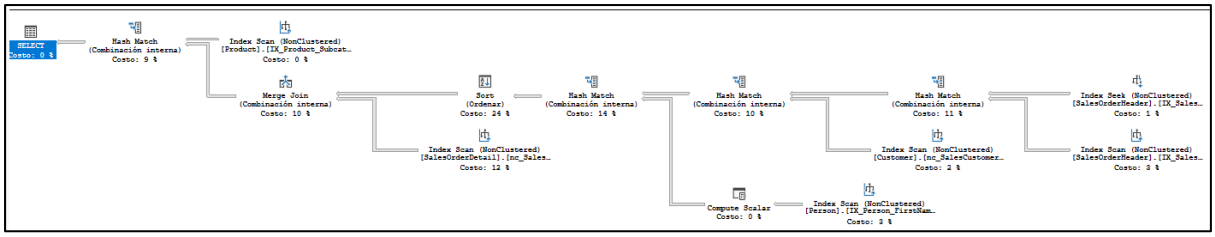


Fig.3 Plan de ejecución estimado de la consulta reescrita

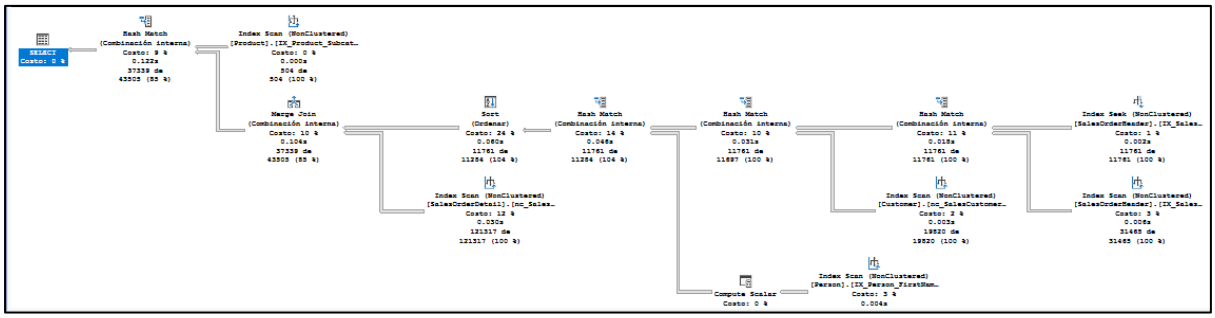


Fig.4 Plan de ejecución real de la consulta reescrita

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
1	2026-03-23	0ms	0ms	303	5% (Index Seek No Agrupado)	Consolidar los índices redundantes en SalesOrderDetail para minimizar la estructura

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
2	2026-03-23	9	15	1193	87 (Búsqueda en índice clúster) -- Operadores con costo	Reescribir la consulta y crear índices

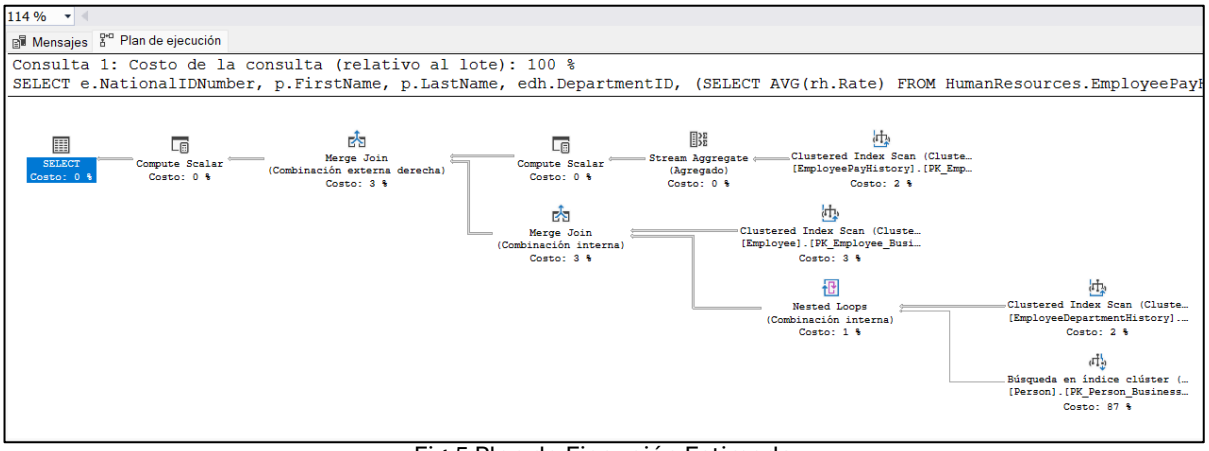


Fig.5 Plan de Ejecución Estimado

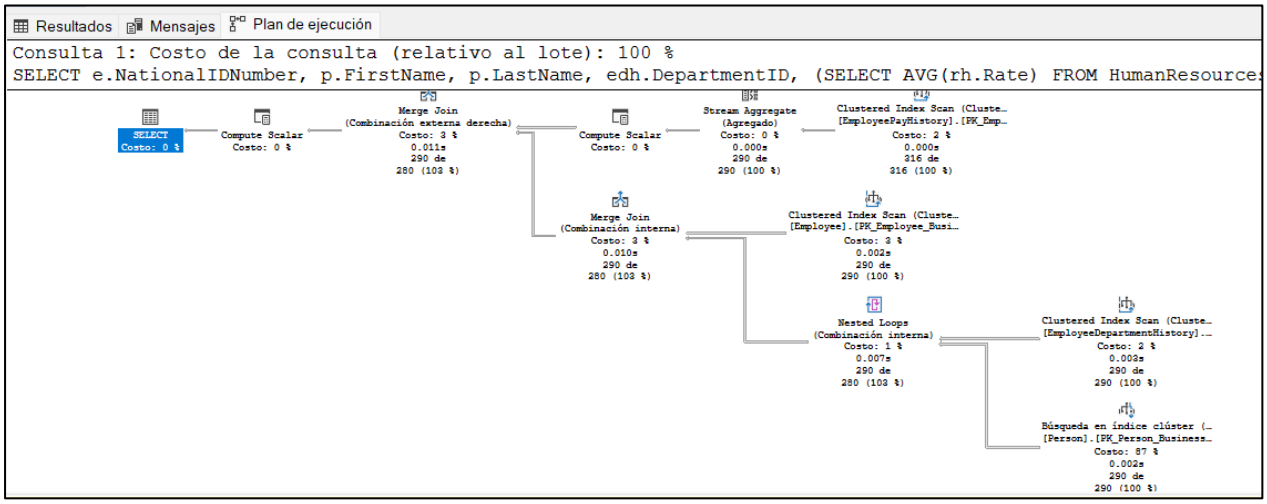


Fig.6 Plan de ejecución real

Consulta reescrita

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
2	2026-03-23	19	15	101	40(HASH MATCH) -- Operadores con costo	Crear índices no agrupados para la columna EndDate y NationalIDNumber

```
SELECT e.NationalIDNumber, p.FirstName, p.LastName, edh.DepartmentID,
AVG(rh.Rate) as PromedioSalario
FROM HumanResources.Employee e
JOIN Person.Person p ON e.BusinessEntityID = p.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory edh ON e.BusinessEntityID =
edh.BusinessEntityID
LEFT JOIN HumanResources.EmployeePayHistory rh
   ON rh.BusinessEntityID = e.BusinessEntityID
WHERE edh.EndDate IS NULL
GROUP BY e.NationalIDNumber, p.FirstName, p.LastName, edh.DepartmentID;
```

La consulta fue reescrita porque al calcular el promedio (AVG) es necesario utilizar un GROUP BY para agrupar correctamente los datos y evitar que el motor procese una cantidad excesiva de registros como ocurría en la consulta original. En la consulta original, el cálculo del promedio tardaba más debido a que no se estaba agrupando la información de forma adecuada. También se agregó un LEFT JOIN con el objetivo de manejar de manera más eficiente los valores NULL, permitiendo descartar o controlar esos registros durante la ejecución. Sin embargo, el cambio en la consulta solo representó una mejora en el plan de ejecución estimado. La verdadera reducción en las lecturas lógicas se logró al crear un índice no agrupado en la tabla Person.Person, utilizando la columna FirstName como clave e incluyendo la columna LastName.

Índice que se creó

```
CREATE NONCLUSTERED INDEX IX_Person_FirstName
ON Person.Person (FirstName)
INCLUDE (LastName)
```

Se podría mejorar el tiempo de duración y el uso de CPU mediante la creación de índices no agrupados sobre las columnas EndDate y NationalIDNumber. Sin embargo, esta optimización no se considera necesaria, ya que el cambio únicamente impacta el tiempo de ejecución, pero no modifica las lecturas lógicas ni el plan de ejecución estimado. Por ello, aunque técnicamente es posible crear más índices para reducir algunos tiempos de respuesta, no siempre es recomendable si la mejora no es significativa o no impacta en los factores más críticos del rendimiento.

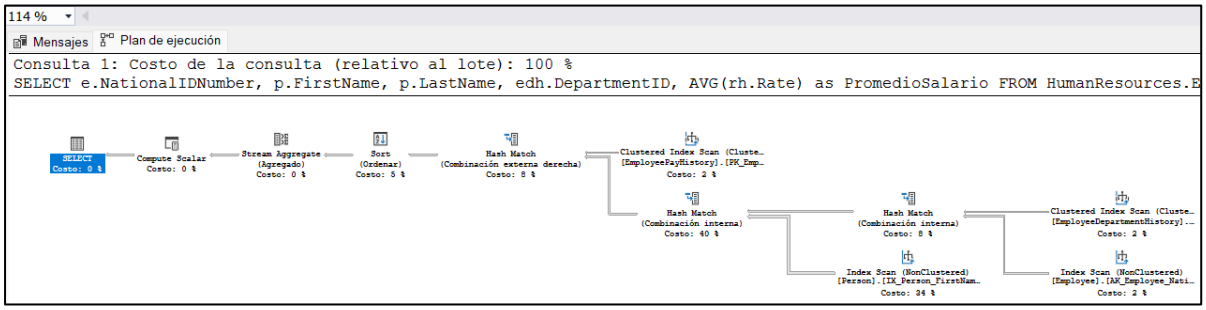


Fig.3 Plan de ejecución estimando de la consulta optimizada

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
3	2026-03-23	0	0	1246	62 (Clustered Index Scan) -- Operadores con costo	Reescribir la consulta y crear índices

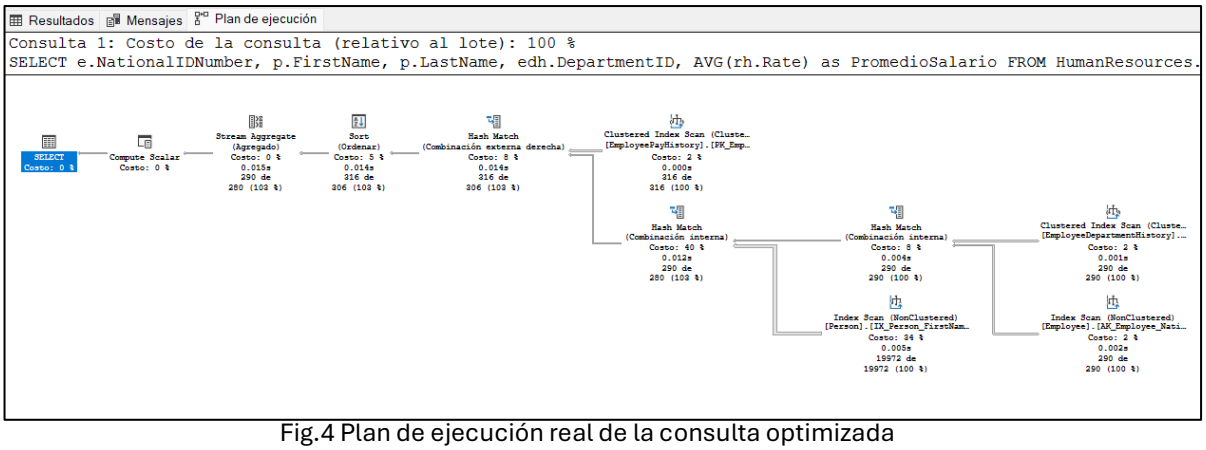


Fig.4 Plan de ejecución real de la consulta optimizada

La consulta fue reescrita ya que en CategoryID marca un error, pues esa columna no pertenece a la tabla Product. Se reescribió de acuerdo a lo que se estaba solicitando en la original

Consulta reescrita

```
WITH categoria as (
 SELECT ps.ProductSubcategoryID
 FROM Production.ProductCategory pc
 JOIN Production.ProductSubcategory ps on ps.ProductCategoryID=Pc.ProductCategoryID
 where pc.ProductCategoryID between 1 and 3
```

```
)
SELECT sod.SalesOrderID, p.ProductID, p.Name
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p ON sod.ProductID = p.ProductID
JOIN categoria c on p.ProductSubcategoryID=c.ProductSubcategoryID
where p.ListPrice>500;
```

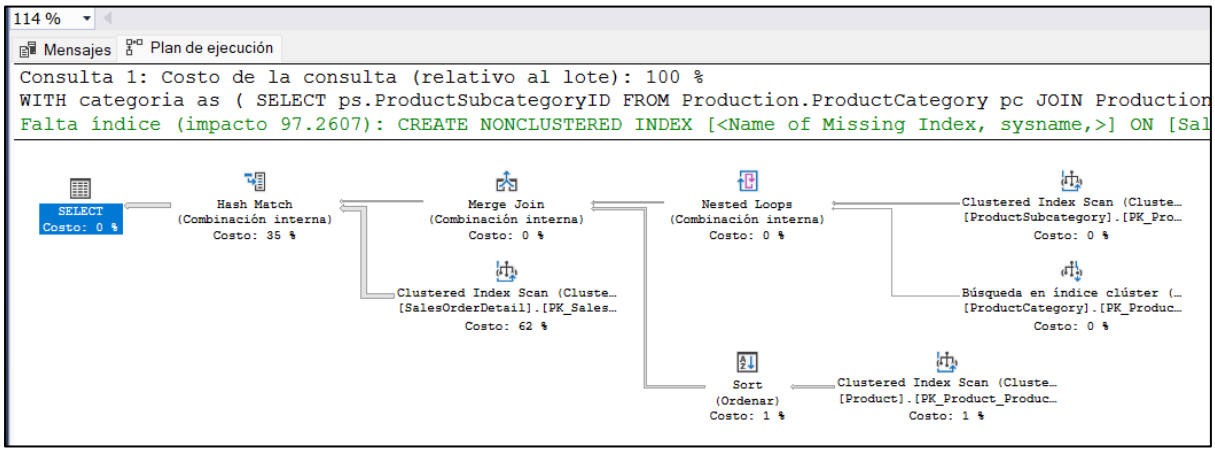


Fig.5 Plan de ejecución estimado de la consulta reescrita

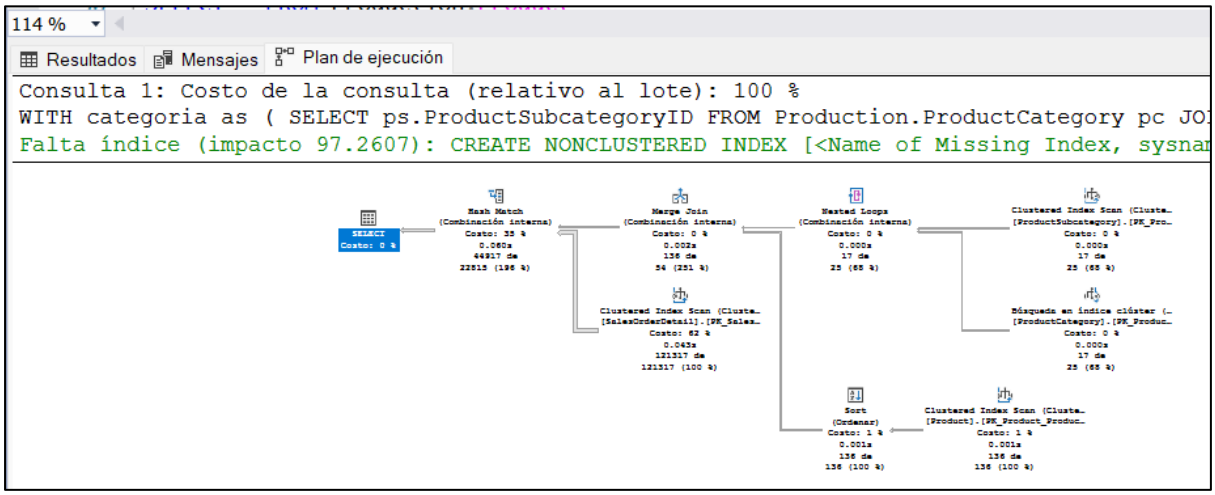


Fig.6 Plan de ejecución real de la consulta reescrita

Primero se creó un índice no agrupado en la columna ProductID, incluyendo SalesOrderID, en la tabla SalesOrderDetail, ya que es la tabla con mayor volumen de registros. Esta tabla estaba generando un número muy alto de lecturas lógicas, debido a que el motor realizaba un recorrido completo tanto en SalesOrderDetail como en Product, lo que incrementaba el costo de la consulta. Con la creación del índice, el plan de ejecución cambió de un Index Scan a un Index Seek, lo que permitió un acceso más eficiente a los datos. Aunque el costo estimado aumentó ligeramente en un 1%, las lecturas lógicas se redujeron significativamente, bajando a 428, lo que representa una mejora importante en el rendimiento real. El segundo índice se creó en la columna ProductCategoryID de la tabla ProductSubcategory. En este caso, el impacto fue menor en el plan de ejecución, ya que el plan estimado prácticamente no cambió. Sin embargo,

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
3	2026-03-23	0	0	428 2	61(Index Seek) -- Operadores con costo	

sí hubo una mejora en las lecturas lógicas, estas pasaron de 50 a 2, lo que demuestra una optimización en el acceso a los datos de esa tabla.

Índices que se crearon

```
CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_ProductID
ON Sales.SalesOrderDetail(ProductID)
INCLUDE (SalesOrderID)

CREATE NONCLUSTERED INDEX IX_ProductSubcategory_ProductCategoryID
ON Production.ProductSubcategory (ProductCategoryID)
```

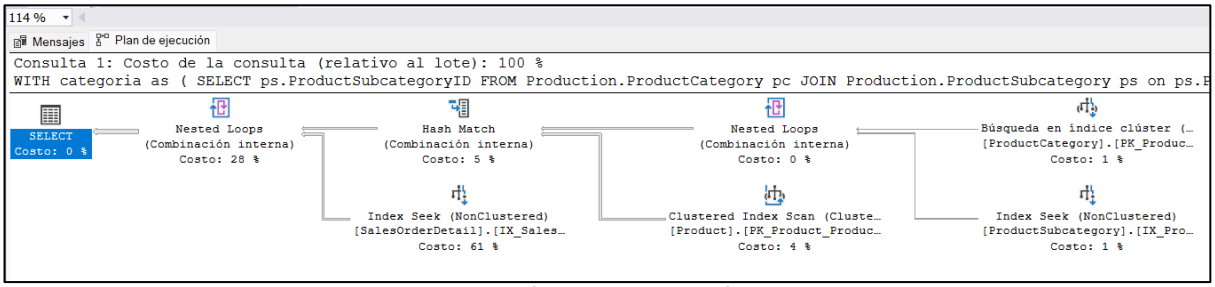


Fig.7 Plan de ejecución estimado con índices agrupados

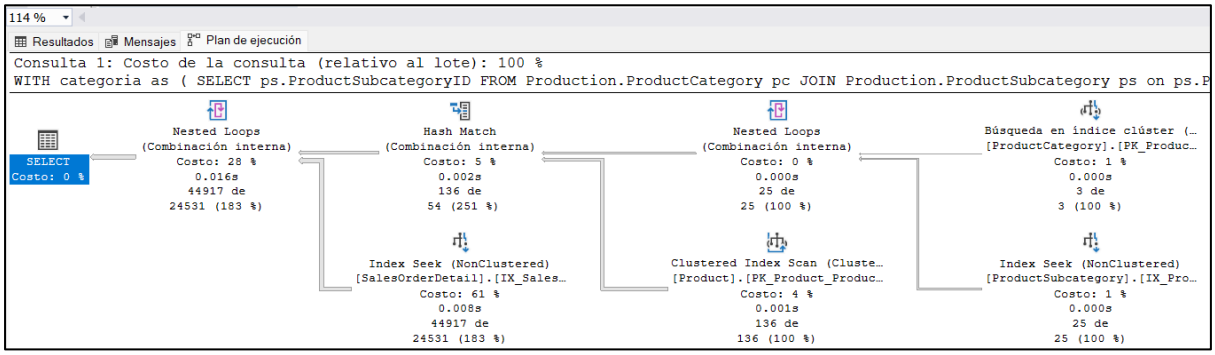


Fig.8 Plan de ejecución real con índices agrupados

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
4	2026-03-23	0	0	1246 684	38 (Clustered Index Scan) -- Operadores con costo	Crear índices

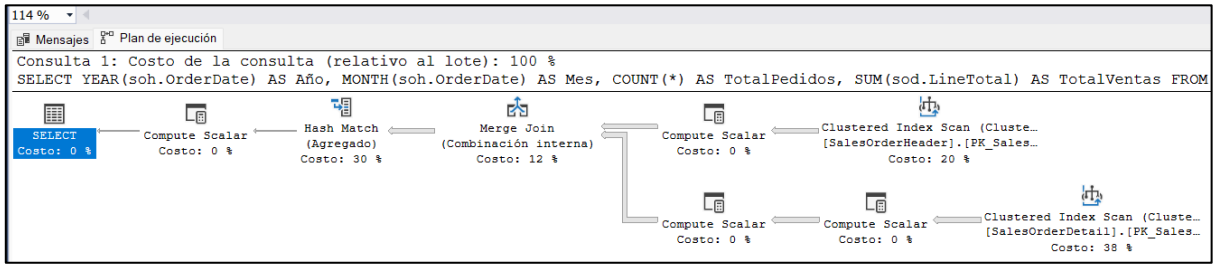


Fig.9 Plan de ejecución estimado de la consulta original

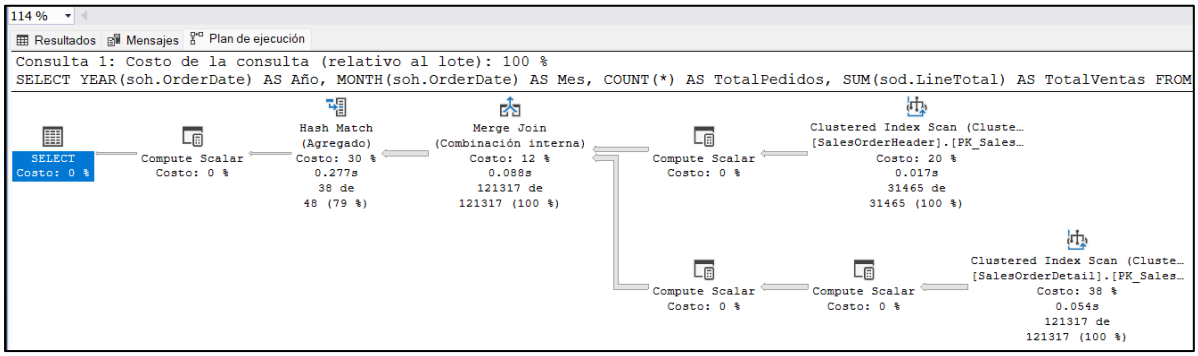


Fig.10 Plan de ejecución real de la consulta original

Se implementó un índice no agrupado en la columna OrderDate, incluyendo la columna SalesOrderID, en la tabla SalesOrderHeader. Esto se realizó porque se trata de una tabla con un gran volumen de datos y el plan de ejecución mostraba que el operador más costoso era un Index Scan sobre esta tabla. Después de agregar el índice, se observó una disminución significativa en las lecturas lógicas tanto en SalesOrderHeader como en SalesOrderDetail, lo que indica una mejora en el acceso a los datos y en el rendimiento general de la consulta. Aunque el plan de ejecución continúa mostrando un Index Scan, esto se debe a que el optimizador considera que, dado el volumen de resultados retornados, sigue siendo la estrategia más adecuada. Sin embargo, el costo estimado disminuyó tras la creación del índice, lo que confirma que el acceso a los datos es ahora más eficiente.

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
4	2026-03-23	0	0	469 73	19 (Clustered Index Scan) -- Operadores con costo	
Índice creado CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_OrderDate ON Sales.SalesOrderHeader (OrderDate) INCLUDE (SalesOrderID)						

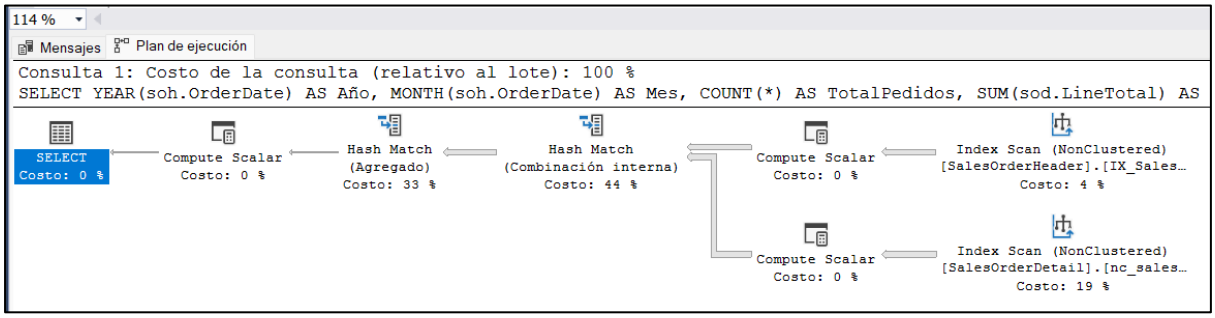


Fig.11 Plan de ejecución estimado con índice creado

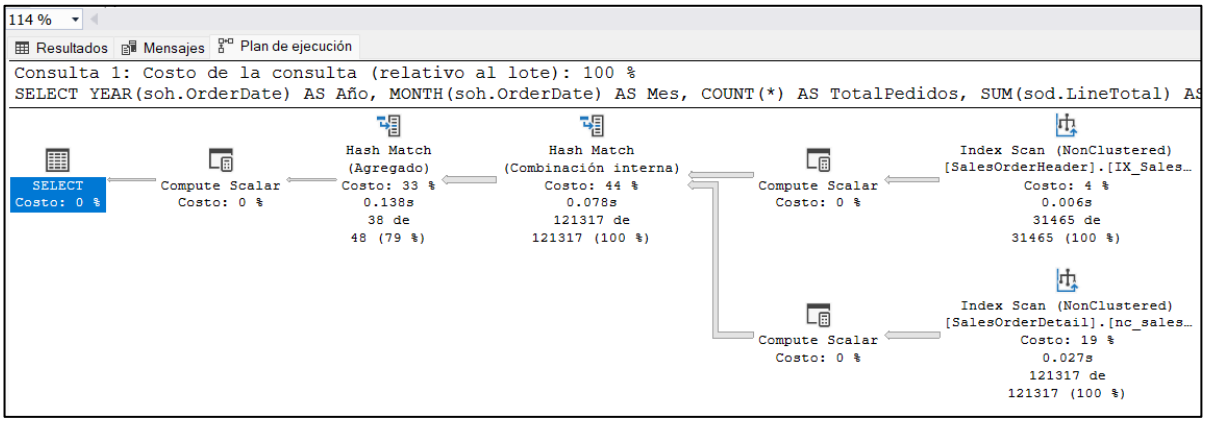


Fig.12 Plan de ejecución real con índice creado

Consulta 6

```
SELECT c.CustomerID, p.FirstName AS Name, COUNT(*) AS TotalPedidos

FROM Sales.Customer c

JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID

JOIN Person.Person p ON c.PersonID = p.BusinessEntityID

WHERE p.FirstName LIKE 'A%' GROUP BY c.CustomerID, p.FirstName;
```

Fig13. Esta consulta se modificó creando un **JOIN** con la tabla **person** para que el atributo name dejara de marcar error

```
CREATE nonclustered index nc_Person

ON Person.Person (FirstName);

Create nonclustered index nc_SalesCustomer

on Sales.Customer (PersonID);
```

Fig14. Se crean índices no agrupados para mejorar el rendimiento de las consultas. El índice en la tabla Person sobre la columna FirstName permite realizar búsquedas más rápidas por nombre, evitando recorridos completos de la tabla. El índice en la tabla **Sales.Customer** sobre la columna PersonID optimiza las operaciones de JOIN entre las tablas Customer y Person, permitiendo localizar los registros relacionados de forma más eficiente.

Consulta ID	Fecha	Duración	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
6	2026-03-22	0ms	0ms	103	Scan bajo (sin operadores costosos)	Crear índice para reducir scans

Fig15. Resultados

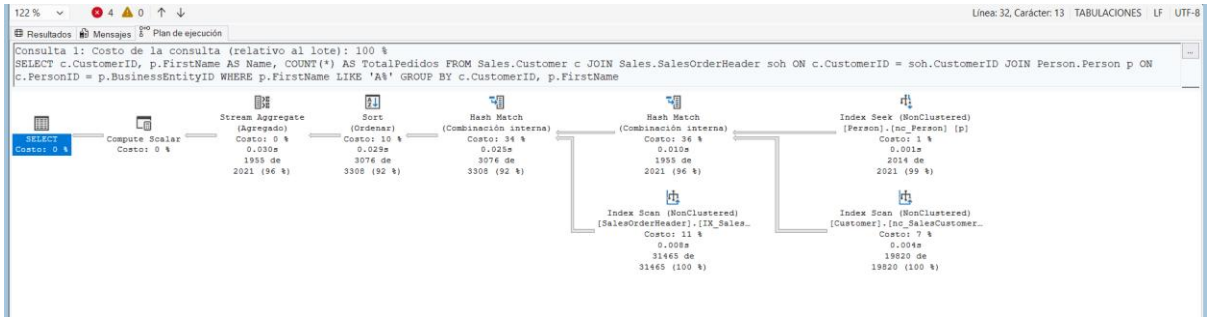


Fig16. Plan de ejecución real

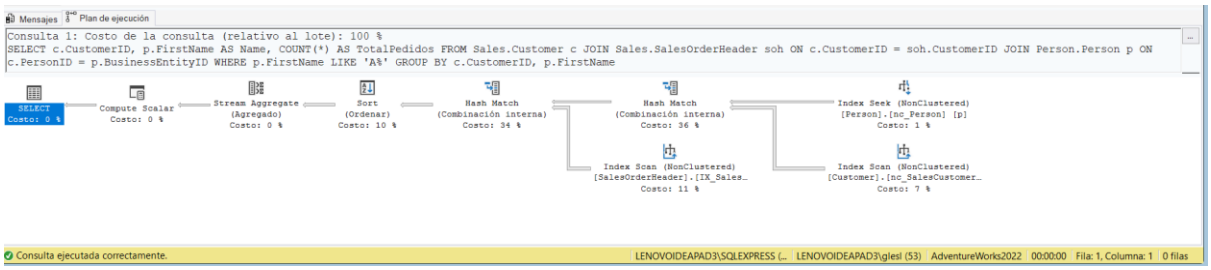


Fig17. Plan de ejecución estimado

Consulta 7:

```
SELECT TOP 100
sod.SalesOrderDetailID, sod.OrderQty, sod.UnitPrice, soh.OrderDate
from ( select top 100 SalesOrderID, ShipDate, OrderDate
from Sales.SalesOrderHeader order by ShipDate DESC )soh
JOIN Sales.SalesOrderDetail sod ON sod.SalesOrderID = soh.SalesOrderID
ORDER BY soh.ShipDate DESC, sod.OrderQty DESC, sod.UnitPrice DESC;
```

Fig 18. En la consulta agregué un **top 100** dentro del **FROM** porque al crear los índices la lectura lógica se redujo bastante pero el plan de ejecución real seguía mostrando el sort:88% significa que la consulta seguía tardando mucho simplemente en ordenar los datos. La razón es que el ORDER BY utiliza columnas de distintas tablas, por lo que SQL Server tarda más en procesar el ordenamiento ya que primero debe realizar el JOIN y después ordenar todo el conjunto resultante. Al aplicar el TOP 100 dentro de la tabla soh, se reduce la cantidad de filas desde el inicio, disminuye los datos que se deben unir y ordenar, mejorando así el rendimiento de la consulta.

Índices creados:

```
create nonclustered index nc_SalesOrderHeader
on Sales.SalesOrderHeader (ShipDate DESC, SalesOrderID)
include(OrderDate);

create nonclustered index nc_SalesOrderDetail
on Sales.SalesOrderDetail (SalesOrderID, OrderQty DESC,UnitPrice DESC)
include(SalesOrderDetailID);
```

Fig 19. Se crean 2 índices no agrupados el índice de **SalesOrderHeader** ayuda a reducir costo de ordenamiento, hacer el JOIN más rápido y evitar lecturas extra. El índice **SalesOrderDetail** ayuda acelerar el JOIN, reducir lecturas y mejorar el order by.

Consulta ID	Fecha	Duración	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
7	2026-03-22	3ms	0ms	347	5(sort)-Nested Loop eficiente	Reducir filas antes del ORDER BY (uso de TOP)

Fig20. Resultados

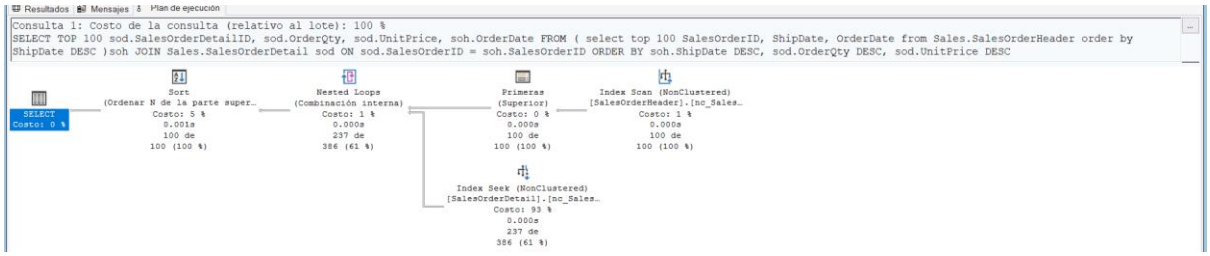


Fig 21. Plan de ejecución real

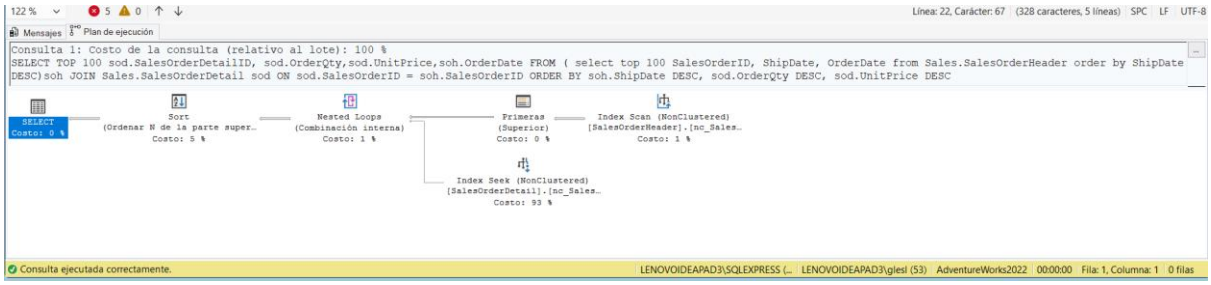


Fig22. Plan de ejecución estimado

Consulta 8:

```
SELECT p.ProductID, p.Name, SUM(sod.OrderQty) AS TotalVendido
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod
   ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh
   ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.OrderDate >= '2014-01-01'
GROUP BY p.ProductID, p.Name
HAVING SUM(sod.OrderQty) > 100;
```

Fig23. Cambié el IN por un JOIN por tres razones sencillas:

1. Para que no me diera error: Con el IN, las cantidades estaban 'atrapadas' dentro del paréntesis y SQL no me dejaba sumarlas. Con el JOIN, junte todo y fue mas sencillo.
2. Por velocidad: El JOIN es más directo. En lugar de que la base de datos dé dos vueltas, con el JOIN hace todo el trabajo una vez y acaba más rápido.
3. Índices: Al usar el JOIN, pude poner un índice que funciona como un atajo. Así, la base de datos ya no tiene que leer miles de filas, sino que va directo a lo que necesita y gasta menos memoria.

Índice creado:

```
CREATE nonclustered index nc_SalesOrderDetail_Op
ON Sales.SalesOrderDetail (SalesOrderID)
INCLUDE (ProductID, OrderQty);
```

Fig24. El plan y las lecturas lógicas bajaron de **1,238 a 303**. O sea, la consulta ahora es **4 veces más ligera**. El detalle de la venta ya no hace un escaneo de toda la tabla, sino que usa el índice de cobertura que hice para ir directo a lo que necesita para sumar y agrupar.

Consulta ID	Fecha	Duración	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
8 clientes	2026-03-22	41ms	16ms	37	Hash Match(36%)	Reducir filas antes del ORDER BY (uso de TOP)
8 ventas	2026-03-23	167ms	63ms	303	Hash Match (52%)	Cambiar `IN` por `JOIN`. Crear **Índice de Cobertura** en `SalesOrderDetail`

Fig26. Resultados

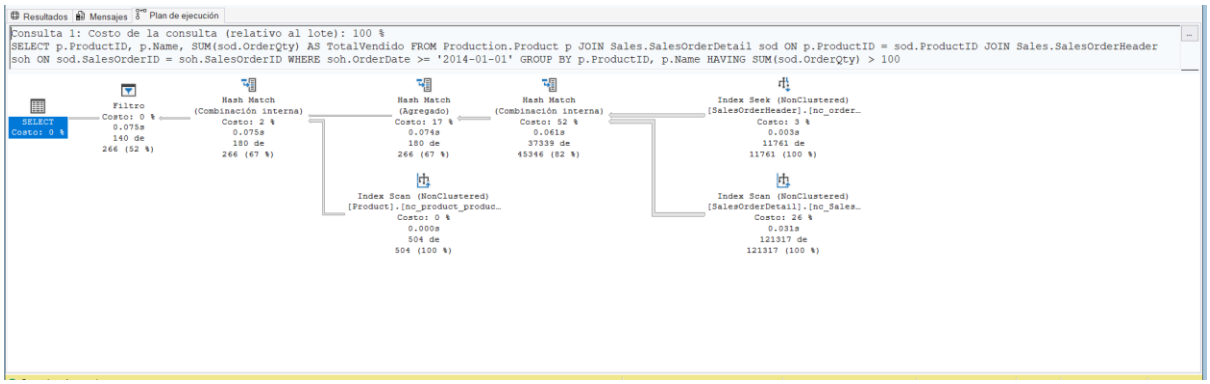


Fig27. Plan de ejecución real

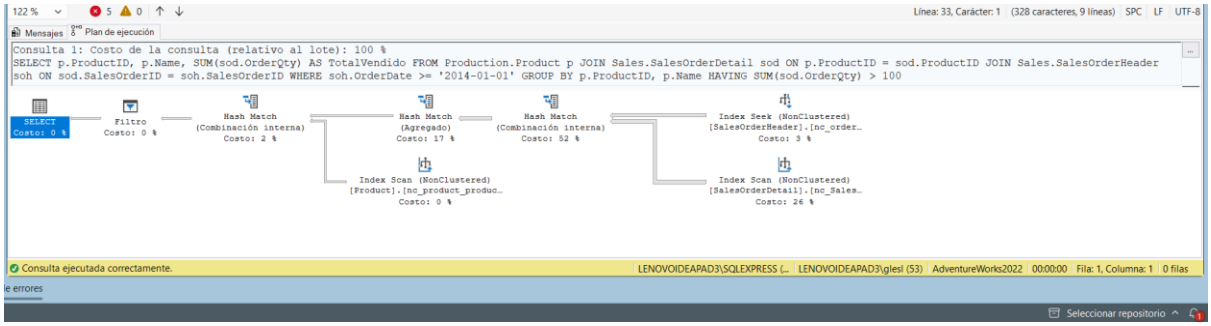


Fig28. Plan de ejecución estimado

Consulta 9

Consulta ID	Fecha	Duración	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
9	2026-03-27	74ms	31ms	1483	Clustered index Scan(53%)	Reescribir la consulta y crear índices

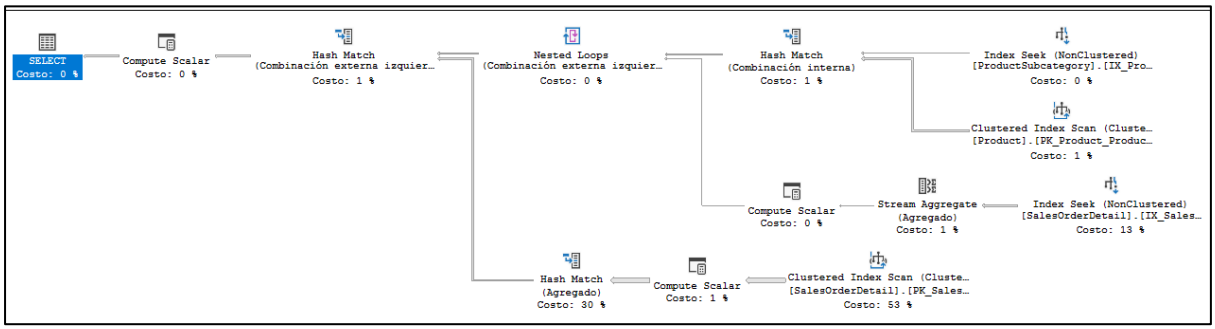


Fig.29 Plan de ejecución estimado

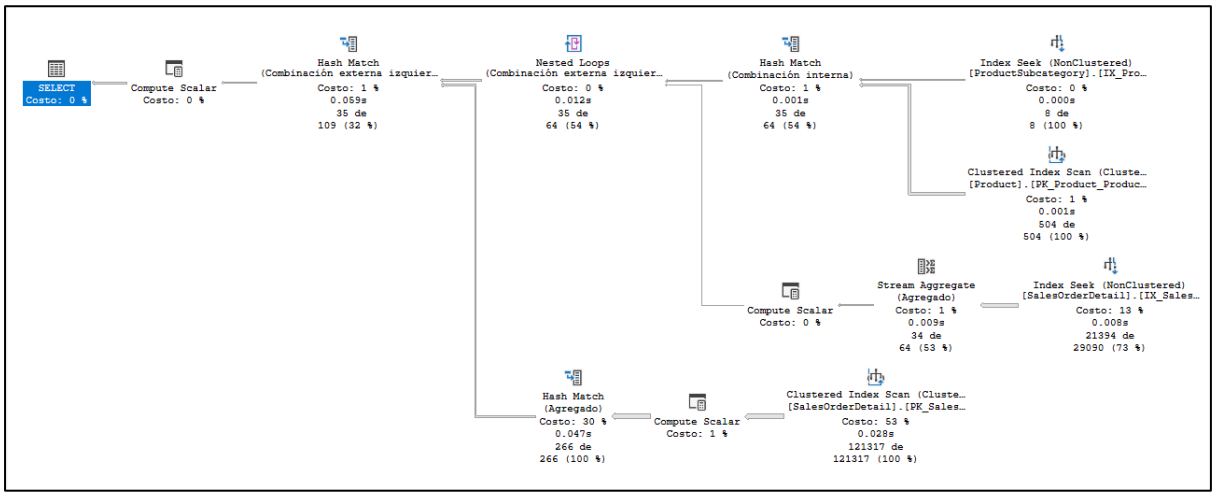


Fig.30 Plan de ejecución real

Consulta Optimizada

```
SELECT p.ProductID, p.Name,
   COUNT(sod.ProductID) AS VecesVendido,
   SUM(sod.OrderQty * sod.UnitPrice) AS Ingresos
FROM Production.Product p
JOIN Production.ProductSubcategory psc
   ON p.ProductSubcategoryID = psc.ProductSubcategoryID
LEFT JOIN Sales.SalesOrderDetail sod
   ON sod.ProductID = p.ProductID
WHERE psc.ProductCategoryID = 3
GROUP BY p.ProductID, p.Name;
```

Índices creados

```
CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_Producto
ON Sales.SalesOrderDetail (ProductID)
INCLUDE (OrderQty, UnitPrice);
CREATE NONCLUSTERED INDEX IX_ProductSubcategory_Category
ON Production.ProductSubcategory (ProductCategoryID)
INCLUDE (ProductSubcategoryID);
CREATE NONCLUSTERED INDEX IX_Product_Subcategory
ON Production.Product (ProductSubcategoryID)
INCLUDE (ProductID, Name);
```

La consulta fue reescrita debido a dentro del SELECT no es eficiente, ya que estas se ejecutan repetidamente por cada fila. Para optimizarla se usó la cláusula GROUP BY, permitiendo realizar los cálculos en una sola operación. Además, se utilizó un LEFT JOIN con el objetivo de incluir también aquellos productos que no han sido vendidos, asegurando así un resultado más completo. Aunque la reescritura de la consulta produjo una mejora en las lecturas, el cambio más significativo se obtuvo al crear índices. En particular, se optimizó la tabla SalesOrderDetail, que es la de mayor tamaño, mediante un índice que incluye las columnas involucradas en las operaciones

(ProductID, OrderQty y UnitPrice). Asimismo, se crearon índices en las tablas ProductSubcategory y Product sobre las columnas utilizadas en los JOIN y filtros.

Consulta ID	Fecha	Duración	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
9	2026-03-27	0ms	14ms	250	Clustered index Scan(53%)	

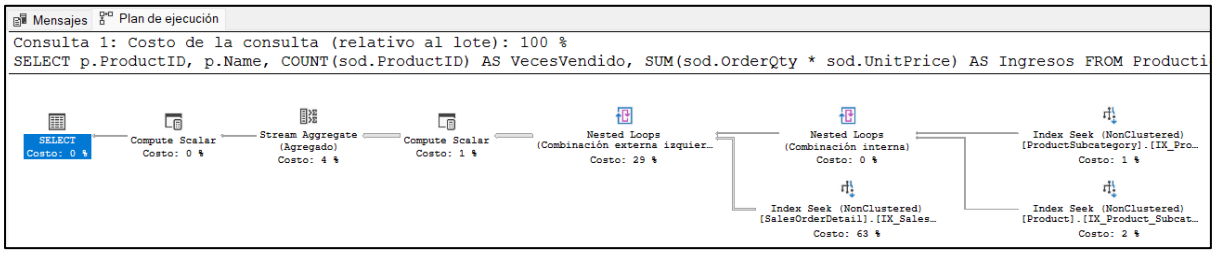


Fig.31 Plan de ejecución estimado de la consulta reescrita

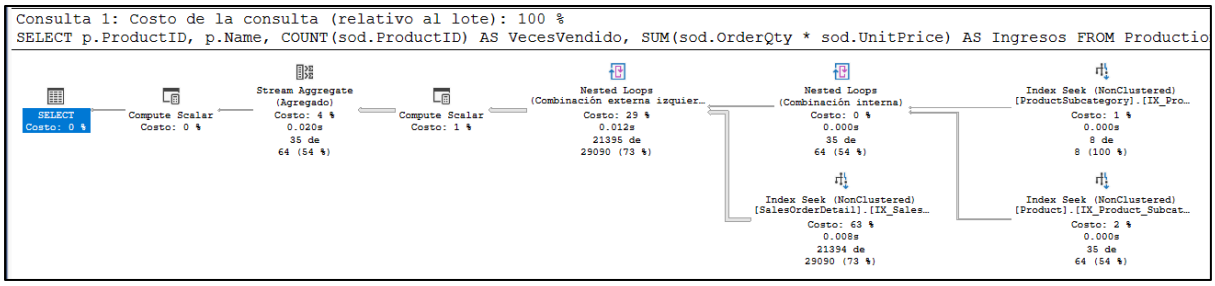


Fig. 32 Plan de ejecución real de la consulta reescrita

Consulta 10

Consulta ID	Fecha	Duración	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
9	2026-03-27	74ms	31ms	1483	Clustered index Scan(53%)	Reescribir la consulta y crear índices

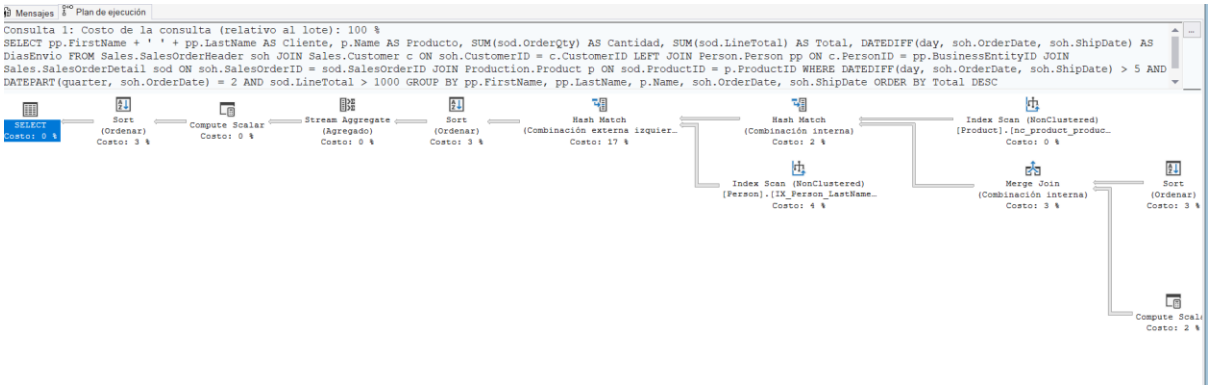


Fig.1 Plan de Ejecución estimado

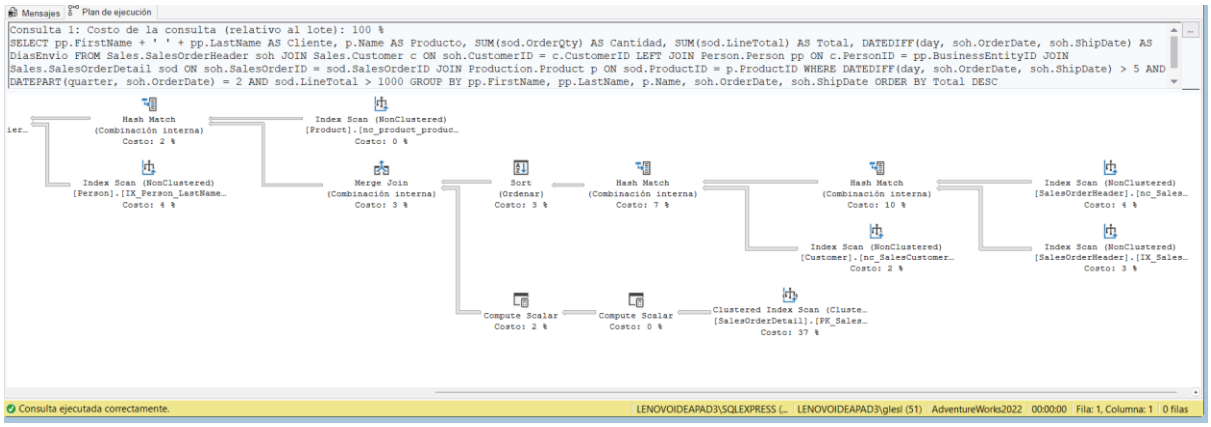


Fig.2 Plan de ejecución estimado

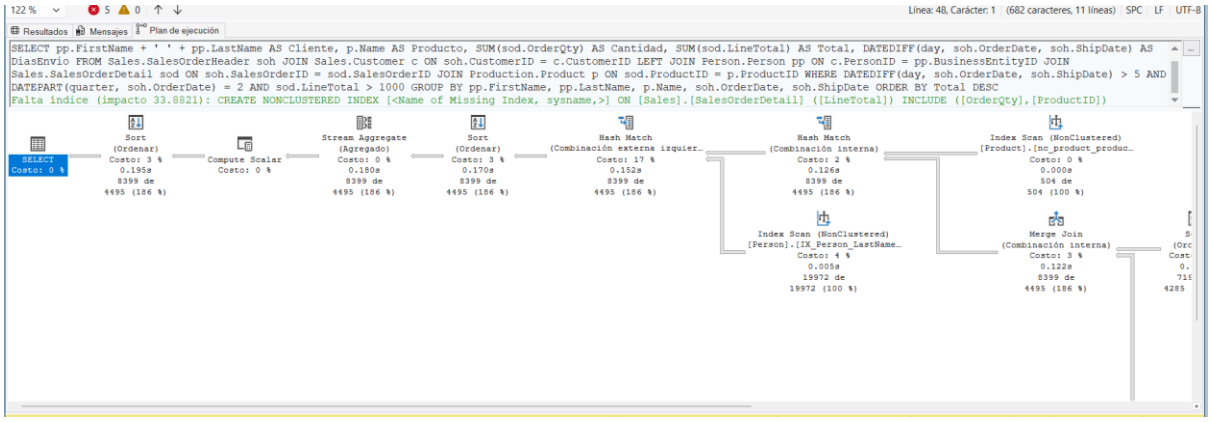
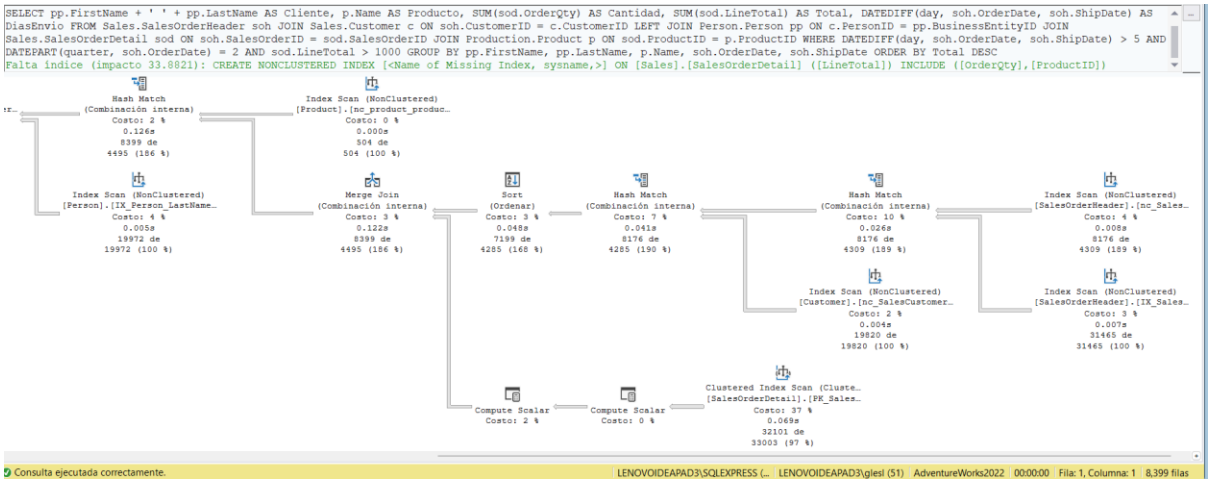


Fig.3 Plan de ejecución real



Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
9	2026-03-23	5	0	152	5% (Index Seek No Agrupado)	Consolidar los índices redundantes en SalesOrderDetail para minimizar la estructura

Fig. 4 Plan de ejecución real

Consulta reescrita

```
SELECT pp.FirstName + ' ' + pp.LastName AS Cliente, p.Name AS Producto,
SUM(sod.OrderQty) AS Cantidad, SUM(sod.LineTotal) AS Total, DATEDIFF(day,
soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh JOIN Sales.Customer c ON soh.CustomerID =
c.CustomerID
LEFT JOIN Person.Person pp ON c.PersonID = pp.BusinessEntityID
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
JOIN Production.Product p ON sod.ProductID = p.ProductID
WHERE DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5
  AND DATEPART(quarter, soh.OrderDate) = 2
  AND sod.LineTotal > 1000
GROUP BY pp.FirstName, pp.LastName, p.Name, soh.OrderDate, soh.ShipDate
ORDER BY Total DESC;
```

La consulta fue reescrita porque la información no saliera repetida por cada día que compraron, agrupé los resultados por Cliente y por Producto. Finalmente, ordené toda la tabla de mayor a menor (DESC) basándome en el total de dinero, para tener a los clientes que más gastaron hasta arriba de la lista.

Índice que se creó

```
CREATE NONCLUSTERED INDEX nc_SalesOrderDetailp

ON Sales.SalesOrderDetail (LineTotal) INCLUDE
(OrderQty,ProductID)
```

Al revisar cómo se estaba procesando mi consulta (viendo el plan de ejecución), me di cuenta de que la base de datos estaba haciendo un esfuerzo innecesario y consumiendo muchos recursos. El problema principal era que, para encontrar las ventas mayores a $1000, el sistema estaba leyendo absolutamente todos los registros de la tabla de detalles de venta (SalesOrderDetail). Era como leer un directorio telefónico completo página por página solo para encontrar a una persona.

La solución: Para arreglar esto y hacer la consulta mucho más rápida, decidí crear un índice nuevo llamado nc_SalesOrderDetailp. Lo configuré para que ordenara los datos basándose principalmente en el total de la compra (LineTotal).

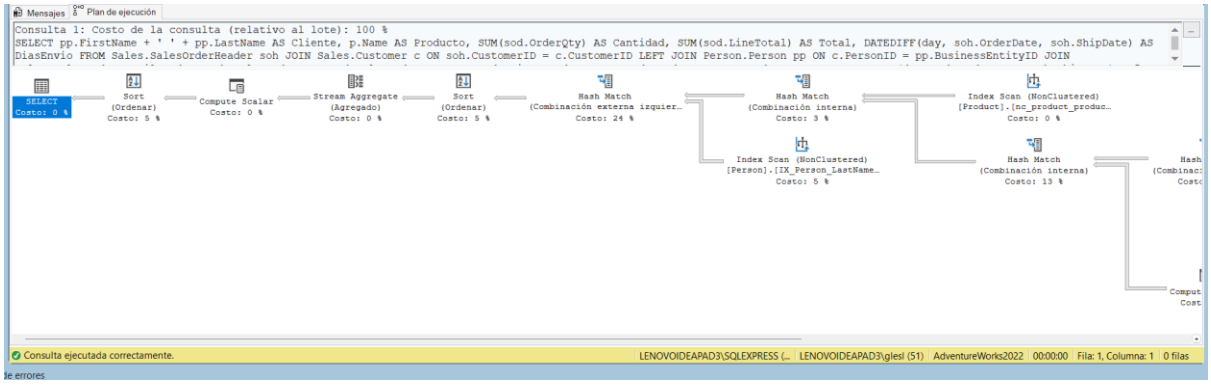


Fig.1 Plan de Ejecución estimado optimizado

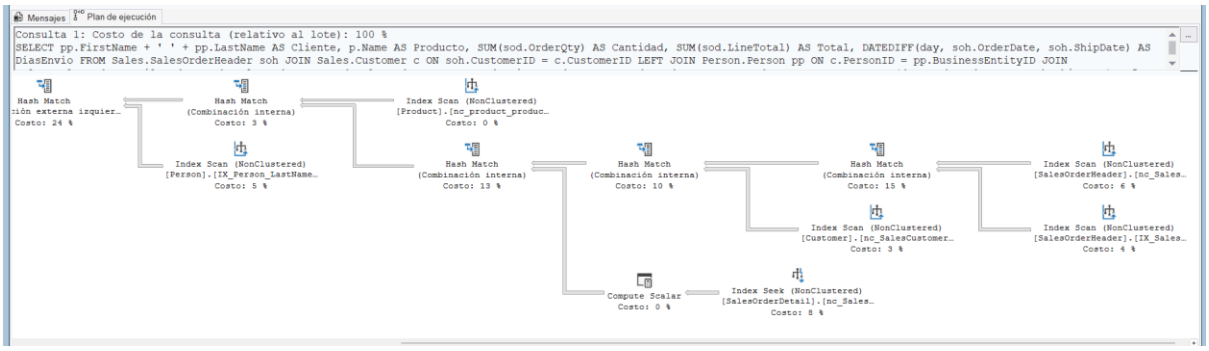


Fig.2 Plan de ejecución estimado optimizado

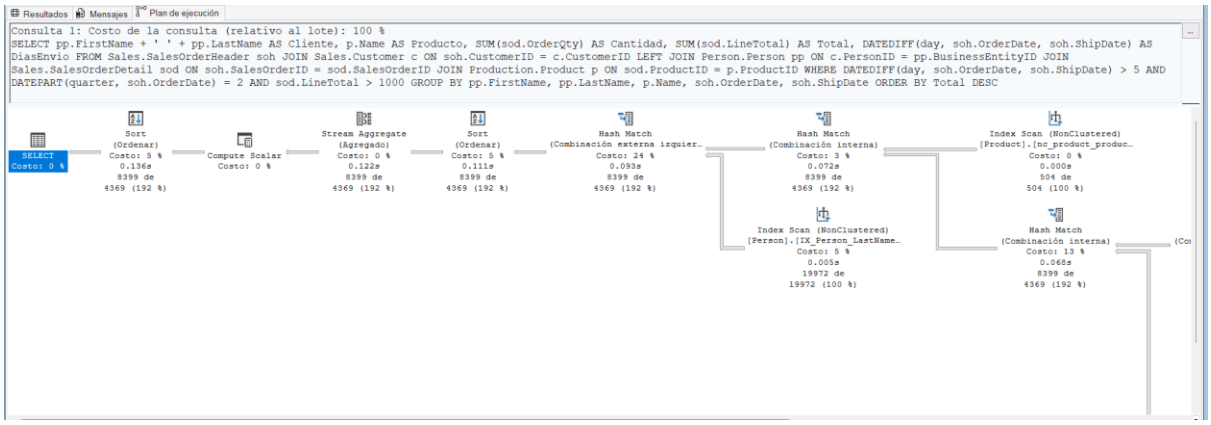


Fig.3 Plan de ejecución real optimizado

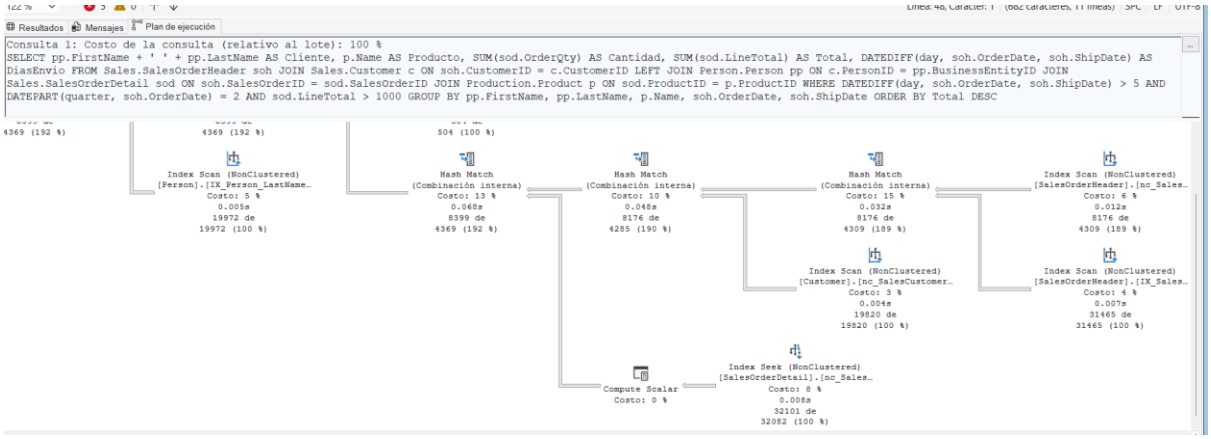


Fig. 4 Plan de ejecución real optimizado